

Yes. If your goal is to **see the Spring/Spring Boot application execute and print the project flow in the console**, there are two different things you can print:

1. **Your source-code files** — print every .java, .xml, .properties, etc. file in project order.
2. **Runtime execution flow** — print messages showing exactly which Spring/Spring Boot components execute and in what order.

For learning Spring/Spring Boot, #2 is much more useful.

1. The execution order you should visualize

For a typical Spring Boot application, the high-level flow is:

```
main()
  ↓
SpringApplication.run()
  ↓
Create SpringApplication
  ↓
Prepare Environment
  ↓
Create ApplicationContext
  ↓
Load configuration
  ↓
Component scanning
  ↓
Bean definitions discovered
  ↓
BeanFactory creates beans
  ↓
BeanPostProcessor processing
  ↓
Dependency injection
  ↓
@PostConstruct
  ↓
InitializingBean / init-method
  ↓
ApplicationContext refreshed
  ↓
ApplicationRunner / CommandLineRunner
  ↓
ApplicationReadyEvent
  ↓
Application is READY
  ↓
HTTP request
  ↓
DispatcherServlet
  ↓
Controller
  ↓
Service
```

↓
Repository
↓
Database
↓
Response

The exact internal sequence is more complicated, because Spring has many lifecycle callbacks and infrastructure beans, but this is the useful application-level execution model.

2. Add execution logging to your project

For example, suppose your project has:

```
src/main/java
├── com.example.demo
│   ├── DemoApplication.java
│   ├── controller
│   │   └── UserController.java
│   ├── service
│   │   └── UserService.java
│   ├── repository
│   │   └── UserRepository.java
│   └── config
│       └── AppConfig.java
```

You can deliberately add console messages at important lifecycle points.

Main class

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("1. main() START");
        System.out.println("=====");

        SpringApplication.run(DemoApplication.class, args);

        System.out.println("=====");
        System.out.println("10. main() END");
        System.out.println("=====");
    }
}
```

Configuration class

```
@Configuration
public class AppConfig {

    public AppConfig() {
        System.out.println("2. AppConfig constructor");
    }

    @Bean
    public MyComponent myComponent() {

        System.out.println("5. Creating MyComponent bean");

        return new MyComponent();
    }
}
```

Component

```
@Component
public class MyComponent {

    public MyComponent() {
        System.out.println("6. MyComponent constructor");
    }

    @PostConstruct
    public void postConstruct() {
        System.out.println("7. MyComponent @PostConstruct");
    }
}
```

Service

```
@Service
public class UserService {

    public UserService() {
        System.out.println("Service constructor");
    }

    @PostConstruct
    public void init() {
        System.out.println("Service @PostConstruct");
    }
}
```

Controller

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {

        System.out.println("Controller constructor");

        this.userService = userService;
    }

    @GetMapping
    public String getUsers() {

        System.out.println("Controller -> getUsers()");

        return "Users returned successfully";
    }
}
```

3. Print the complete application lifecycle

A better approach is to use Spring Boot's lifecycle events.

```
@Component
public class ApplicationLifecycleLogger {

    @EventListener
    public void onStarting(ApplicationStartingEvent event) {
        System.out.println(">>> 01 ApplicationStartingEvent");
    }

    @EventListener
    public void onEnvironmentPrepared(ApplicationEnvironmentPreparedEvent event) {
        System.out.println(">>> 02 ApplicationEnvironmentPreparedEvent");
    }

    @EventListener
    public void onContextInitialized(ApplicationContextInitializedEvent event) {
        System.out.println(">>> 03 ApplicationContextInitializedEvent");
    }

    @EventListener
    public void onPrepared(ApplicationPreparedEvent event) {
```

```

        System.out.println(">>> 04 ApplicationPreparedEvent");
    }

    @EventListener
    public void onStart(ApplicationStartedEvent event) {
        System.out.println(">>> 05 ApplicationStartedEvent");
    }

    @EventListener
    public void onReady(ApplicationReadyEvent event) {
        System.out.println(">>> 06 ApplicationReadyEvent");
    }
}

```

However, there is an important detail: **very early Spring Boot events occur before normal application beans are available**, so an `@EventListener` component cannot necessarily observe every early event. For complete startup tracing, Spring Boot's startup instrumentation is better.

4. Use Spring Boot's StartupStep/ApplicationStartup mechanism

You can configure startup tracking:

```

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {

        SpringApplication application =
            new SpringApplication(DemoApplication.class);

        application.setApplicationStartup(
            new BufferingApplicationStartup(2048)
        );

        application.run(args);
    }
}

```

Then you can inspect startup steps.

For example:

```

@Component
public class StartupLogger {

    private final ApplicationStartup applicationStartup;
}

```

```
public StartupLogger(ApplicationStartup applicationStartup) {  
    this.applicationStartup = applicationStartup;  
}  
}
```

For practical debugging, however, **Spring Boot's DEBUG logging is often easier.**

5. Enable Spring Boot DEBUG logging

In:

src/main/resources/application.properties

add:

debug=true

Then run:

mvn spring-boot:run

or:

./mvnw spring-boot:run

You will get considerably more information about auto-configuration and startup.

You can also enable specific Spring packages:

```
logging.level.org.springframework=DEBUG  
logging.level.org.springframework.boot=DEBUG  
logging.level.org.springframework.context=DEBUG  
logging.level.org.springframework.beans=DEBUG  
logging.level.org.springframework.web=DEBUG
```

For Hibernate/JPA:

```
logging.level.org.hibernate=DEBUG  
logging.level.org.springframework.orm.jpa=DEBUG
```

6. Print bean creation order

If your real objective is:

"Show me every Spring bean as it is created."

you can use a BeanPostProcessor.

```
@Component
public class BeanExecutionLogger
    implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(
        Object bean,
        String beanName) {

        System.out.println(
            "BEFORE INITIALIZATION -> "
            + beanName
            + " -> "
            + bean.getClass().getName()
        );

        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(
        Object bean,
        String beanName) {

        System.out.println(
            "AFTER INITIALIZATION -> "
            + beanName
            + " -> "
            + bean.getClass().getName()
        );

        return bean;
    }
}
```

This produces output such as:

```
BEFORE INITIALIZATION    -> userService -> com.example.demo.service.UserService
AFTER INITIALIZATION     -> userService -> com.example.demo.service.UserService

BEFORE INITIALIZATION    -> userController -> com.example.demo.controller.UserController
```

AFTER INITIALIZATION -> userController -> com.example.demo.controller.UserController

Important: this is not necessarily the complete creation order of *all* beans. Some infrastructure and lifecycle behavior happens outside what this simple logger observes.

7. Print constructor → injection → initialization

For learning Spring, this is one of the best techniques.

```
@Service
public class UserService {

    public UserService() {
        System.out.println(
            "UserService -> Constructor"
        );
    }

    @PostConstruct
    public void postConstruct() {

        System.out.println(
            "UserService -> @PostConstruct"
        );
    }

    @PreDestroy
    public void preDestroy() {

        System.out.println(
            "UserService -> @PreDestroy"
        );
    }
}
```

And:

```
@RestController
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {

        System.out.println(
            "UserController -> Constructor"
        );
    }
}
```

```

        this.userService = userService;
    }

    @GetMapping("/users")
    public String users() {

        System.out.println(
            "UserController -> users()"
        );

        return "OK";
    }
}

```

You can then see:

UserService -> Constructor
 UserService -> @PostConstruct

UserController -> Constructor

and when you call:

GET /users

you see:

UserController -> users()

8. Print the HTTP request execution flow

For Spring MVC applications, add an interceptor.

```

@Component
public class ExecutionInterceptor
    implements HandlerInterceptor {

    @Override
    public boolean preHandle(
        HttpServletRequest request,
        HttpServletResponse response,
        Object handler) {

        System.out.println(
            "1. HTTP Request received: "
            + request.getMethod()
            + " "

```

```

        + request.getRequestURI()
    );

    System.out.println(
        "2. Handler: "
        + handler
    );

    return true;
}

@Override
public void postHandle(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    ModelAndView modelAndView) {

    System.out.println(
        "3. Controller execution completed"
    );
}

@Override
public void afterCompletion(
    HttpServletRequest request,
    HttpServletResponse response,
    Object handler,
    Exception exception) {

    System.out.println(
        "4. HTTP Request completed"
    );
}
}

```

Register it:

```

@Configuration
public class WebConfig
    implements WebMvcConfigurer {

    private final ExecutionInterceptor interceptor;

    public WebConfig(ExecutionInterceptor interceptor) {
        this.interceptor = interceptor;
    }

    @Override
    public void addInterceptors(

```

```
        InterceptorRegistry registry) {  
    registry.addInterceptor(interceptor);  
}  
}
```

Now:

GET /users

can produce:

1. HTTP Request received: GET /users
 2. Handler: com.example.demo.controller.UserController#getUsers()
 3. Controller execution completed
 4. HTTP Request completed
-

9. Print Controller → Service → Repository

For a complete application flow, add logging to each layer.

Controller

```
@GetMapping("/{id}")  
public String getUser(@PathVariable Long id) {  
  
    System.out.println(  
        "CONTROLLER -> getUser(" + id + ")"  
    );  
  
    return userService.getUser(id);  
}
```

Service

```
public String getUser(Long id) {  
  
    System.out.println(  
        "SERVICE -> getUser(" + id + ")"  
    );  
  
    return userRepository.findUser(id);  
}
```

Repository

```
public String findUser(Long id) {  
  
    System.out.println(  
        "REPOSITORY -> findUser(" + id + ")"  
    );  
  
    return "User-" + id;  
}
```

The console becomes:

```
HTTP REQUEST  
  ↓  
CONTROLLER -> getUser(100)  
  ↓  
SERVICE -> getUser(100)  
  ↓  
REPOSITORY -> findUser(100)  
  ↓  
DATABASE  
  ↓  
REPOSITORY RETURN  
  ↓  
SERVICE RETURN  
  ↓  
CONTROLLER RETURN  
  ↓  
HTTP RESPONSE
```

10. Use a sequence number

If you want the console to show the **exact order of your own logging statements**, create a centralized logger.

```
@Component  
public class ExecutionTracker {  
  
    private final AtomicInteger counter =  
        new AtomicInteger(0);  
  
    public void log(String message) {  
  
        int number = counter.incrementAndGet();  
  
        System.out.println(  
            String.format(  
                "%03d | %s",  
                number,  

```

```

        message
    )
};
}
}

```

Then inject it:

```

@Service
public class UserService {

    private final ExecutionTracker tracker;

    public UserService(ExecutionTracker tracker) {
        this.tracker = tracker;

        tracker.log(
            "UserService constructor"
        );
    }

    @PostConstruct
    public void init() {

        tracker.log(
            "UserService @PostConstruct"
        );
    }
}

```

Controller:

```

@RestController
public class UserController {

    private final ExecutionTracker tracker;
    private final UserService userService;

    public UserController(
        ExecutionTracker tracker,
        UserService userService) {

        this.tracker = tracker;
        this.userService = userService;

        tracker.log(
            "UserController constructor"
        );
    }
}

```

```

@GetMapping("/users")
public String getUsers() {

    tracker.log(
        "UserController.getUsers()"
    );

    return "SUCCESS";
}
}

```

Console:

```

001 | UserService constructor
002 | UserService @PostConstruct
003 | UserController constructor
004 | Application started
005 | HTTP GET /users
006 | UserController.getUsers()
007 | HTTP response generated

```

This makes the execution sequence extremely easy to understand.

11. If you literally mean "print all source-code files"

If by "print all project code" you mean:

Print the contents of every source file in the console in project execution order.

then you can create a utility that reads the project files.

For example:

```

public class ProjectCodePrinter {

    public static void main(String[] args) throws Exception {

        Path root =
            Paths.get("src/main/java");

        Files.walk(root)
            .filter(Files::isRegularFile)
            .filter(path ->
                path.toString().endsWith(".java"))
            .sorted()
            .forEach(path -> {

```

```

System.out.println(
    "\n=====
");

System.out.println(
    "FILE: " + path
);

System.out.println(
    "=====
");

try {

    Files.lines(path)
        .forEach(System.out::println);

} catch (IOException e) {

    e.printStackTrace();
}
});
}
}

```

But there is a **fundamental difference**:

File order

≠

Compilation order

≠

Bean creation order

≠

Runtime execution order

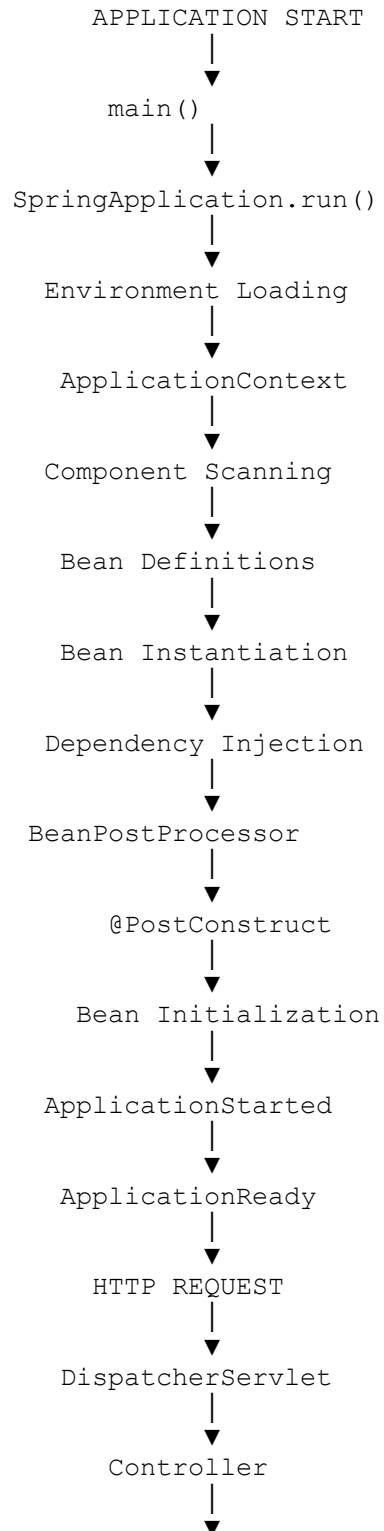
≠

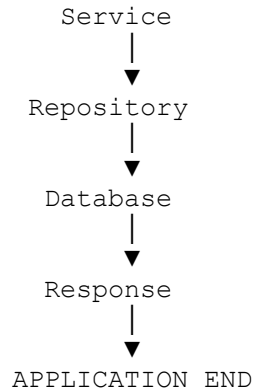
HTTP request execution order

So you should **not try to determine Spring execution order simply by sorting .java files**.

12. Best approach for your Spring + Spring Boot projects

Since you want to learn the **complete sequence from project start to project end**, I recommend building the projects with an explicit execution-tracing layer:





For your earlier **separate Spring and Spring Boot projects**, I would make the console output even more systematic by assigning every significant operation a number such as:

SPRING EXECUTION TRACE

=====

```
001 JVM started
002 main() entered
003 Application configuration loaded
004 ApplicationContext created
005 BeanDefinition registered
006 Component scanned
007 Bean instantiated
008 Dependency injected
009 BeanPostProcessor.beforeInitialization()
010 @PostConstruct executed
011 BeanPostProcessor.afterInitialization()
012 ApplicationContext refreshed
013 Application started
014 Application ready
015 HTTP request received
016 DispatcherServlet invoked
017 Controller invoked
018 Service invoked
019 Repository invoked
020 Database operation
021 Repository returned
022 Service returned
023 Controller returned
024 HTTP response sent
```

=====

APPLICATION END

=====

That gives you a **visual execution trace**, rather than merely dumping source code.